ASSIGNMENT 1 - TASK 12: Re-evaluate and Adapt the Design

Name: K.V.N.D.P.SAI RUPESH | Register Number: 1972260102 | Bloom's Level: K5 - Evaluate | Marks: 8

Source code: ParkingSystemV2.java (same folder). Output screenshots: see the screenshot files in this folder.

Decision: MODIFY the original design. The class structure is kept and extended; two classes received small changes.

A. Impact Analysis (2 marks)

What has changed

- Two new vehicle categories: e-bikes (flat fee) and delivery vans (double the car rate from 9 AM to 5 PM).

- Visitors can buy a one-day pass (before, only monthly passes for students and staff existed).

- The facility grows from 3 to 5 floors, and each floor has a different number of slots.

Which classes, methods and data structures are affected

Part of the design	Affected?	Reason
Vehicle.calculateFee(minutes)	Yes (extended)	A van's fee depends on the time of day, so a time-aware method calculateFee(entryTime, minutes) was added.
ParkingFacility.createVehicle()	Yes (modified)	Two new cases: E_BIKE and DELIVERY_VAN.
ParkingFacility.exit()	Yes (modified)	The boolean hasPass was replaced by a Pass object so that day and monthly passes can be checked.
TwoWheeler, Car, ElectricCar, Chargeable, ParkingTicket	No	They work as they are.
slots array	No change in code	Already a jagged array.

Why a different number of slots per floor affects the array design

A rectangular array such as new String[5][50] gives every floor the same number of slots, which wastes memory and invents slots that do not exist. A jagged array (new ParkingTicket[5][] with a separate length for each floor) fits the real layout. Because allocate() always loops up to slots[f].length, it needs no change.

Classes modified rather than extended

Two: Vehicle (one new method added; existing code untouched) and ParkingFacility (factory switch and exit signature).

B. Evaluation of the Original Design (2 marks)

- **Can new vehicle types be added only by writing new subclasses?** Almost. EBike and DeliveryVan are new subclasses. The one extra step is adding a case to the createVehicle() switch.

- **Are pricing rules spread across many classes or kept in one place?** Each rule sits in its own vehicle class (the rate in ratePerHour(), special rules in an overridden calculateFee()). This is easy to find, but the rules are spread over several classes and there is no single pricing class. The pass rule is in one place: the Pass interface.

- **Can the existing tested classes remain unchanged?** Yes for five classes (TwoWheeler, Car, ElectricCar, Chargeable, ParkingTicket). The Task 8 results (Rs 30, Rs 90, Rs 210) were tested again and are the same.

Conclusion: the original design is appropriate. Inheritance and polymorphism absorbed both new vehicle types, so a full redesign is not needed.

C. Adaptation (2 marks)

Revised class structure

Class / interface	Status	Purpose
Vehicle (abstract)	Modified	Added calculateFee(entryTime, minutes) which by default calls calculateFee(minutes).
TwoWheeler, Car, ElectricCar	Unchanged	Old categories.
EBike extends Vehicle	New	Overrides calculateFee(): flat Rs 20 per visit.
DeliveryVan extends Car	New	Overrides calculateFee(entryTime, minutes): each billable hour that starts from 9:00 up to (not including) 17:00 costs 2 x car rate.
Pass (interface)	New	covers(entryTime, exitTime) tells if the stay is free.
MonthlyPass, DayPass	New	Implement Pass. DayPass is valid only on one day.
ParkingFacility	Modified	Two new factory cases; exit() takes a Pass object; slots stay a jagged array (2, 3, 1, 4, 2).

What changed and why

- **Van fee rule:** the rate of each billable hour is decided by the minute at which that hour starts. A van entering at 16:30 for 90 minutes pays 60 (hour starting 16:30) + 30 (hour starting 17:30) = Rs 90.

- **One-day pass:** made as a Pass interface, so a future pass type (for example a weekly pass) only needs one new class.

- **Fee calculation uses integer arithmetic:** hours = (minutes + 59) / 60, so every started hour is charged.

Test results for the new requirements

Test	Expected	Actual	Result
E-bike 30 min (flat Rs 20)	20.0	20.0	PASS
E-bike 300 min (flat Rs 20)	20.0	20.0	PASS
Van 10:00, 120 min (all peak)	120.0	120.0	PASS
Van 20:00, 120 min (off-peak)	60.0	60.0	PASS
Van 16:30, 90 min (crosses 5 PM)	90.0	90.0	PASS
Van starts exactly 17:00, 60 min	30.0	30.0	PASS
Van starts exactly 9:00, 60 min	60.0	60.0	PASS
Visitor car, valid one-day pass	0.0	0.0	PASS
Visitor car, pass for another day	60.0	60.0	PASS
Student car, monthly pass	0.0	0.0	PASS
Two-wheeler 130 min (old)	30.0	30.0	PASS
Car 130 min (old)	90.0	90.0	PASS
EV 130 min + 10 kWh (old)	210.0	210.0	PASS
6th car goes to 1-slot floor 2	[2, 0]	[2, 0]	PASS
12th car takes last slot	[4, 1]	[4, 1]	PASS

Test	Expected	Actual	Result
13th car, facility full	null	null	PASS
Unknown vehicle type	rejected	rejected	PASS
Duplicate registration	rejected	rejected	PASS
Day total (hand-calculated)	320.0	320.0	PASS

Hand calculation of the day total: e-bike 300 min = Rs 20; van 10:00 for 120 min = 60 + 60 = Rs 120; van 16:30 for 90 min = 60 + 30 = Rs 90; car with valid one-day pass = Rs 0; car 130 min = 3 x 30 = Rs 90. Total = 20 + 120 + 90 + 0 + 90 = **Rs 320**, which matches the program output. Program summary: Passed: 19 Failed: 0.

D. Evidence-Based Defence (2 marks)

Measure	Result
Existing classes that had to change	2 of 7 (Vehicle, ParkingFacility)
Existing classes left unchanged	5 of 7 (TwoWheeler, Car, ElectricCar, Chargeable, ParkingTicket)
New classes / interfaces added	5 (EBike, DeliveryVan, Pass, MonthlyPass, DayPass)
Test results	19 of 19 passed, including the old Task 8 results

Ease of adding a further vehicle type

Write one new subclass and add one case to createVehicle(). Fee rules live inside the subclass, and no if-else on vehicle type exists anywhere in the fee code.

Trade-offs

- **Kept the design (modify):** cheap, low risk, and old tests still pass. The cost is that createVehicle() still needs one edit for each new type.

- **Full redesign (rejected):** a PricingRule class for all prices would put every rule in one place, but it would mean rewriting classes that already work and are tested, for no benefit in this case.

- **Limitation:** a van's rule is written inside DeliveryVan. If many time-based rules appear, a separate PricingRule abstraction will become worthwhile.

Final decision: MODIFY. The evidence supports it: only 2 classes changed, 5 stayed unchanged, both new categories and the one-day pass work, and all 19 tests passed.